🍴 ChefCoach

A Duolingo-Style Cooking Coach Agent

Skill Progression + Personalized Coaching + RLHF-Inspired Feedback

EECS 6895 Midterm Project

Team Members:
Peiyan Sun (ps3558) | Rui Lin (rl3445) | Shengbo Yi (sy3302)

Tech Stack:
Gemini 2.5 Flash Lite (Online) | Qwen 2.5:3B (Offline)
Streamlit | ChromaDB | SQLite | Sentence-Transformers

7	Pipeline Modules
20K	Recipe Corpus
4	Eval Methods
100%	vs Baseline

Why ChefCoach?

The Problem

Existing cooking apps only do recipe search.
They give you a recipe and leave you alone.

No skill tracking across sessions
No personalized difficulty adjustment
No structured learning progression
No feedback-driven adaptation

Our Solution

An AI agent that teaches cooking as a structured curriculum, like Duolingo.

skills → practice → feedback → level-up
Maintains learner profile across sessions
RAG retrieval for real recipe context
RLHF-inspired adaptive feedback loop

What Makes ChefCoach an AI Agent (not just a chatbot):

1 Planning

Curriculum planner selects next skill

2 Tool Use

RAG retrieval from 20K recipe corpus

3 Memory

Persistent profile across sessions

4 Adaptation

RLHF-inspired feedback loop

System Architecture

7 modules | 3-phase pipeline | SQLite + ChromaDB + JSON

7 Pipeline Modules

1. **UI Input**
   Goal, time, ingredients, diet
2. **Profile Manager**
   Structured memory (JSON)
3. **Skill Assessor**
   Quiz → skill_vector + level
4. **Curriculum Planner**
   Skill Graph → next lesson
5. **RAG Retriever**
   Chroma + RRF → recipe packets
6. **Coach Generator**
   LLM → steps + fixes + quiz
7. **Feedback → Update**
   RLHF-inspired loop

Phase 1: Input → Profile → Assessment

User goal ("learn stir-fry"), time, ingredients, equipment, dietary constraints
UserProfile: skill_vector (8 dims), taste_prefs, constraints, equipment, history[]
Lightweight assessment: short quiz + self-report → estimated skill_vector + level

Phase 2: Curriculum → RAG → Coach Generation

Choose next target skill: prerequisites + weakest-skill strategy
RAG: chunk by title/ingredients/steps → retrieve top-k → reassemble by recipe_id
Embedding: all-MiniLM-L6-v2 (384-dim) → ChromaDB cosine search + RRF fusion
LLM prompt = learner profile + curriculum state + retrieved context (in-context learning)

Phase 3: Feedback → Profile Update (RLHF-Inspired)

Feedback: rating, difficulty, notes ("too spicy/too bland"), completion status
Update: skill[s] = clip(skill[s] + lr × confidence × delta, 0, 1)
Emphasis: inspired by RLHF concept, not training reward model/PPO

Storage: UserProfile.json | InteractionMemory (SQLite) | Vector DB (ChromaDB) | skills.json

RAG Pipeline + RRF Fusion

How ChefCoach retrieves and assembles recipe knowledge

1. Data Indexing

20K Food.com Recipes
Indexed in ChromaDB with
all-MiniLM-L6-v2 embeddings
(384-dim sentence vectors)

Chunking Strategy:
Each recipe split into 3 fields:
title, ingredients, steps
Stored with recipe_id metadata
for downstream reassembly

2. Retrieval + RRF

Two Parallel Retrieval Sources:

Source A: Recipe chunks
→ semantic search by user query

Source B: Technique snippets
→ 53 curated Q&A entries

Reciprocal Rank Fusion:
score = $\Sigma\ 1/(k + rank_i)$
Merges evidence from both

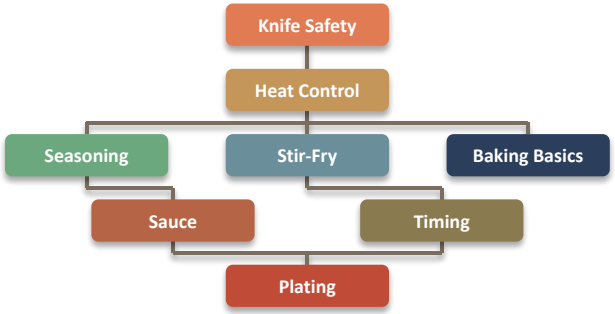
3. Packet Reassembly

Stitching Back Full Context:
Retrieved chunks are grouped
by recipe_id, then reassembled
into complete recipe packets:
title + ingredients + steps
+ rating + tags

→ **Coach LLM Prompt:**
Learner profile
+ curriculum state
+ fused recipe packets
= personalized coaching

Curriculum & Feedback Loop

8-Skill Prerequisite Graph



Strategy: Weakest-Skill-First — lowest-score skill whose prereqs met

RLHF-Inspired Feedback Loop

User feedback after each session:

- 👍 or 👎, difficulty (1-5),
- Taste notes, completion status

Update rule (not PPO, rule-based):

skill[s] += lr × confidence × delta

confidence = EMA(α, old_conf, |delta|)

Why rule-based, not true RLHF?

Small data (<100 entries), interpretable,
stable for current project. Upgrade: LoRA + DPO

XP & Level Gamification

Duolingo-style per-skill progression:

XP = base_xp + difficulty_bonus + streak_mult

6 levels: Novice → Beginner → ... → Master

Thresholds: 0, 100, 300, 600, 1000 XP

Visual progress in dashboard radar chart

Learner Clustering (C1–C4)

Rule-based clustering for personalization:

C1: Quick & Easy (time < 30min)

C2: Adventurous (diverse cuisines)

C3: Health-focused (diet constraints)

C4: Technique-driven (skill-focused)

Cluster influences lesson + recipe filter

Streamlit Demo

Onboarding
Quiz, diet, equipment, goal & time budget

→

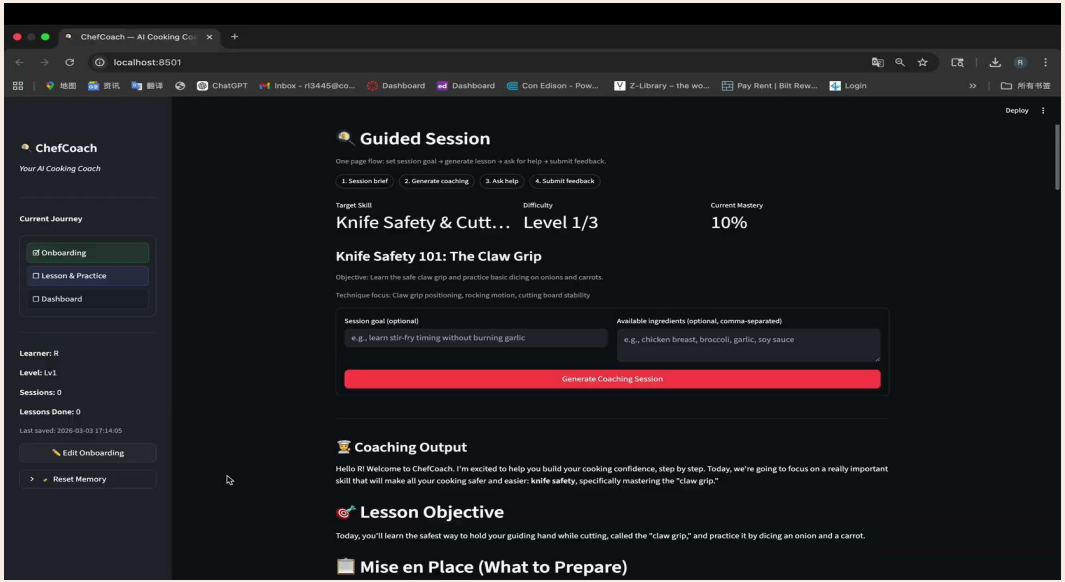
Guided Session
RAG recipe + step-by-step coaching + technique Q&A

→

Feedback
Rating, difficulty, notes, completion ⟶ profile update

→

Dashboard
Skill radar, XP & levels, history & next lessons



1:05 Demo Video (Gemini 2.5 Flash Lite)

Key Adaptation:
How the System Learns

Session 1 (Beginner)

Onboarding ⟶ Heat Control lesson
Level 1/3, 10% mastery
I'm Stuck: "how do I know when the pan is hot enough?"

↓

Session 2 (Adapted)

Level 2/3, mastery 10% ⟶ 17%
Skill radar & XP visibly increased
Feedback loop confirmed working

Evaluation Results

4 automated methods × 2 LLMs (Gemini + Qwen) = rigorous, reproducible evaluation

A Pairwise LLM-as-Judge (24 scenarios, rubric: personalization / difficulty / clarity / safety / teaching)

Comparison	Gemini 2.5 Flash Lite	Qwen 2.5:3B (Ollama)	Result
vs No-Memory Baseline	24/24 = 100%	24/24 = 100%	Full system wins
vs Profile-Only	8/24 = 33%	2/24 = 8%	By design*

B Curriculum Progression Test

Metric	Result
Prerequisite violations	0 / 24
Pass rate	100%
Avg skill gain / session	0.055

C Grounding & Safety Check

Metric	Gemini	Qwen 2.5:3B
Grounding Score	0.875	0.846
Safety Pass Rate	100%	100%
Recipe verification	Pass	Pass

D User Study Template (Optional)

15-question survey: perceived learning, satisfaction, usability, "would cook again"

Status: Template ready. Prioritized automated eval (A/B/C) for reproducible evidence.

Findings & Future Work

Key Findings

100% Win vs Baseline, 33% vs Memory-Only
By design: single-round lr × conf × delta produces small incremental changes. Value compounds over multiple sessions.

Gemini > Qwen 3B: Model Capacity Matters
Eval A win rate: 33% vs 8%
Grounding: 0.875 vs 0.846
3B model struggles with nuanced personalization — same pipeline, fair test.

Rule-Based RLHF: A Deliberate Choice
Small data (<100 entries), interpretable, stable for current project. Clear upgrade path to LoRA + DPO when data grows.

Future Work (Limitations + Improvements)

Multi-Round Evaluation
5–10 session trajectories to measure compounding skill improvement.

Real User Study
5–10 participants, 2–3 real cooking sessions with perceived learning survey.

Larger Offline LLM
Qwen-14B / Llama-8B to close the online-offline quality gap.

Toward True RLHF
LoRA + DPO once dataset reaches 500+.

Expand Skill Graph
8 → 15+ skills with richer prerequisite chains and difficulty curves.

Conclusions

Full system beats baseline 100%. The 33% gap vs memory-only is expected. Small per-session updates add up over time.

Thank You

Q&A